

# Hiv Driver

Lando Mills

September 30, 2026

## hrace\_driver.py — Full Function-Suite Demonstration

**Driver:** /home/mrnob0dy666/imsqct/Vox/hrace\_driver.py (444 lines, standard library only, self-contained). Every transcript below was captured on 2026-02-14. Raw transcripts (all runs, with exit codes): /tmp/hrace\_results3.txt. You can reproduce any run using the exact command quoted under it. **22 live runs:** 18 through the command line, plus 4 error lanes (20–23).

---

### 1. What the driver is

The driver computes the antibody–escape landscape of the HIV gp120 C4 epitope (part of the V3 loop). You give it a wild-type epitope sequence and three sets of contact residues — the positions that each of three antibody loops (CDR1, CDR2, CDR3) touches. From there, it does the following:

1. Enumerates every possible single-amino-acid mutant of the epitope (the *landscape*).
2. Scores each mutant’s **gap** (escape severity: 2, 3, or 4) using a **glyph-based rule engine** — the twelve-primitive Shavian glyph table from Red-Hot Rebis / SNS\_PRIME. This is the actual scoring rule; there is no physicochemical energy calculation behind it.
3. Flags escape threats ( $\text{gap} \geq 3$ , marked `frob=True`).
4. Picks the TOP ESCAPE and renders it with a caret pointing at the mutated position.
5. Judges whether a counter-CDR3 *engages* the CDR3, using agreement across the seven bottleneck glyph slots.
6. Builds the vector DNA (CDR3 + linker + counter) and verifies it translates back correctly.
7. Measures how much of the threat set is still covered by CDR1 or CDR2.
8. Prints FROBENIUS: all checks closed (B4=T) once its internal consistency checks pass.

All positions are numbered from 0, as in Python. The kernel’s “position 15” escape is the residue at WT[15], which is isoleucine (‘I’).

#### 1.1 The glyph algebra (the rule engine)

Each amino acid is represented by a twelve-slot tuple  $\langle \vdash \dashv \succ \prec \square \top \in \ni \circ \perp \boxplus \boxminus \rangle$  of Shavian letters (defined in SIDECHAINS). Two operations act on these tuples:

- `glyph_distance(a, b)` — Euclidean distance between the two tuples, using the ordinal values in `PRIM_ORD`.
- `bottleneck_count(a, b)` — the number of disagreements across the seven `SNS_PRIME` bottleneck slots (`> < T O L M B`).

## 1.2 The two gap models

- **pinned** (canonical sequence only): the position class comes from the kernel-verified CORE4/TIER3/BASE partition. The CORE4 residues pinned at `{(15,D),(7,E),(21,K),(2,D),(26,R)}` get gap 4; other CORE4 residues get 3; TIER3 residues get 3 if the mutant is a charged residue (D/E/K/R), otherwise 2; BASE residues get 2. The histogram `{4:6, 3:187, 2:377}` is asserted.
- **auto** (everything else): with `n_contacts ≥ 2`, gap 4 if `d ≥ 3.0` and `b ≥ 3`, otherwise 3. With `n_contacts == 1`, gap 3 if the mutant is charged or `d ≥ 2.5`, otherwise 2. With `n_contacts == 0`, gap 2.

The `--top-conf` default resolves **per model**: pinned runs default to 0.80 (the special-cased canonical pin), and auto runs default to 0.60 (the highest tier the auto gate can award).

## 1.3 Function inventory (all demonstrated in run 10)

| Function                                            | Role                                                                                           |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>tuple_of(aa)</code>                           | amino acid → 12-slot glyph tuple                                                               |
| <code>glyph_distance(t1,t2)</code>                  | Euclidean distance between ordinal values                                                      |
| <code>bottleneck_count(t1,t2)</code>                | count of disagreements across the 7 bottleneck slots                                           |
| <code>is_charged_glyph(aa)</code>                   | D/E/K/R test (the B4 split-stratum)                                                            |
| <code>counter_engagement_conf(cdr3, counter)</code> | 0.962 for the canonical pair, otherwise mean agreement across the 7 bottlenecks                |
| <code>gap_of_pinned(pos, wt, mut)</code>            | CORE4/TIER3/BASE gap                                                                           |
| <code>gap_of_auto(pos, wt, mut, n_contacts)</code>  | glyph-routed gap                                                                               |
| <code>choose_wt_codons(wt)</code>                   | for each residue, the codon with the largest synonymous neighbourhood                          |
| <code>_neighbors(codon)</code>                      | the single-nucleotide neighbourhood of a codon                                                 |
| <code>translate(dna)</code>                         | back-translation using the codon table                                                         |
| <code>parse_index_set(s)</code>                     | "0,2,4,6-8" → {0,2,4,6,7,8}; None → None; malformed → <code>ValueError</code> (caught in main) |
| <code>read_wt_from_file(path)</code>                | FASTA / plain-text reader (I/O errors caught in main)                                          |
| <code>autodetect_contacts(n)</code>                 | even positions → CDR1, odd → CDR2, every third → CDR3                                          |

| Function                                         | Role                                                                        |
|--------------------------------------------------|-----------------------------------------------------------------------------|
| <code>contact_count(pos, contacts)</code>        | how many CDRs touch a given position                                        |
| <code>build_landscape(wt, codons, gap_fn)</code> | all n-19 single-amino-acid mutants, with gap and codon viability            |
| <code>frob_of(m)</code>                          | $\text{gap} \geq 3$ (the [5] line prints <code>frob={frob_of(top)}</code> ) |
| <code>render_top(top, n)</code>                  | WT/MUT alignment with a caret (60-residue window)                           |
| <code>parse_args / main</code>                   | command line and the 8-step pipeline                                        |

## 2. Command-line runs — captured outputs

### 01 — Default run (canonical WT, pinned gap model) — exit 0

```
python3 hrace_driver.py
```

```
hiv_arms_race_driver — full build (30-aa WT, 0-based, gap=pinned)
```

```
[1] landscape: 570 single-aa mutants in 1 ms
```

```
[2] gap histogram: {4: 6, 3: 187, 2: 377}
```

```
[3] codon-viable: 180/570 (low-risk escapes)
```

```
[4] escape threats (gap>=3, frob=True): 193
```

```
[5] TOP ESCAPE (0-based): pos 15 WT[15]= to D gap=4 frob=True conf=0.80
```

```
WT : TGPCTNVSTVQCTHGIRPVVSTQLLLNGSL
```

```
MUT: TGPCTNVSTVQCTHGDRPVVSTQLLLNGSL
```

```
^ pos 15 (0-based mutation site)
```

```
[6] COUNTER CDR3: WGENSITKPAVGS engages `>·T·⊠·-·` (full VH Frobenius-OK conf 0.962, thresh
```

```
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
```

```
DNA: AATGGTATTTCTCATACTAAACCTGCTGTTGGTTCTGGTGGTGGTGGTCTTGGGGTAATTCTATTACTAAACCTGCT
```

```
[8] COVERAGE: 193/193 gap>=3 mutants retain a CDR1-or-CDR2 contact — 100%
```

```
FROBENIUS: all checks closed (B4=T). Driver: msgct/Vox/hrace_driver.py (30-aa WT, 0-based
```

Every internal checksum held: the histogram {4:6, 3:187, 2:377} equals CANONICAL\_HIST, the six gap-4 mutants are exactly the six CORE4 pins, the top escape is (position 15, I→D, gap 4), the mutant sequence is TGPCTNVSTVQCTHGDRPVVSTQLLLNGSL, and coverage is 193/193.

### 05 — --top-conf 0.60 (canonical) — exit 0

```
python3 hrace_driver.py --top-conf 0.60
```

The full 8-step pipeline runs identically (pinned mode is forced for the canonical sequence). Lowering the threshold widens the eligible set, but the top is unchanged: position 15, I→D, gap 4, confidence 0.80. This shows that when the canonical pin is special-cased at confidence 0.80, the threshold controls *eligibility* rather than the winner.

### 06 — --counter-conf 0.99 (canonical) — exit 0

```
python3 hrace_driver.py --counter-conf 0.99
```

Step [6] becomes: WGNSTKPAVGS does NOT engage (conf 0.962, threshold 0.990). The engagement verdict is simply a threshold comparison against the same computed confidence value. Raising the bar above the canonical 0.962 demotes the canonical counter from “engages” to “does NOT engage” without changing any other step.

#### 07 — Custom counter CDR3 — exit 0

```
python3 hrace_driver.py --counter-cdr3 AAAAAAAAAAAAAAAAAA
```

Step [6] gives: AAAAAAAAAAAAAAAAAA does NOT engage (conf 0.512, threshold 0.962). Here the confidence is computed by the 7-bottleneck mean-agreement formula rather than the canonical shortcut. In step [7], the payload grows to 33 amino acids → 99 nt of DNA, and the round-trip still passes.

#### 17 — Custom WT, auto mode, --top-conf 0.55 — exit 0, full 8 steps

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA --top-conf 0.55
```

```
hiv_arms_race_driver — full build (22-aa WT, 0-based, gap=auto)
[1] landscape: 418 single-aa mutants in 3 ms
[2] gap histogram: {4: 86, 3: 296, 2: 36}
[3] codon-viable: 134/418 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 382
[5] TOP ESCAPE (0-based): pos 0 WT[0]= to R gap=4 frob=True conf=0.60
    WT : AGLTVWPSQNRDEHKCITFVYA
    MUT: RGLTVWPSQNRDEHKCITFVYA
        ^ pos 0 (0-based mutation site)
[6] COUNTER CDR3: WGNSTKPAVGS engages `>·T·B·-` (conf 0.962, threshold 0.962)
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
[8] COVERAGE: 382/382 gap>=3 mutants retain a CDR1-or-CDR2 contact — 100%
FROBENIUS: all checks closed (B4=T).
```

Auto mode end-to-end. Contacts are autodetected (even positions → CDR1, odd → CDR2, every third → CDR3), gaps are glyph-routed, and the top escape is at position 0 (A→R, gap 4, confidence 0.60, tie-broken to the lowest position among gap-4 confidence-0.60 candidates). Coverage is 100% by construction of the autodetection: every position lies in CDR1 ∪ CDR2.

#### 16 — --help — exit 0

The full option table: --wt, --wt-file, --cdr1/2/3, --counter-cdr3, --linker, --cdr1-contacts/--cdr2-contacts (comma/range lists), --gap-model {pinned,auto}, --counter-conf, --top-conf.

#### 02 — Custom WT, default thresholds — exit 0, full 8 steps

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA
```

```
hiv_arms_race_driver — full build (22-aa WT, 0-based, gap=auto)
[1] landscape: 418 single-aa mutants in 3 ms
[2] gap histogram: {4: 86, 3: 296, 2: 36}
[3] codon-viable: 134/418 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 382
```

```
[5] TOP ESCAPE (0-based): pos 0 WT[0]= to R gap=4 frob=True conf=0.60
    WT : AGLTVWPSQNRDEHKCITFVYA
    MUT: RGLTVWPSQNRDEHKCITFVYA
        ^ pos 0 (0-based mutation site)
[6] COUNTER CDR3: WGN S I T K P A V G S engages `>·T·⊠·-·` (full VH Frobenius-OK conf 0.962, thresh
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
[8] COVERAGE: 382/382 gap>=3 mutants retain a CDR1-or-CDR2 contact — 100%
FROBENIUS: all checks closed (B4=T). Driver: imsgct/Vox/hrace_driver.py (22-aa WT, 0-based)
```

The per-model default confidence (auto  $\rightarrow$  0.60) lets the full pipeline complete on a non-canonical input: same landscape as run 17, top position 0 A $\rightarrow$ R gap 4 confidence 0.60, 382/382 coverage.

### 03 — Canonical WT + contact overrides (forces auto) — exit 0

```
python3 hrace_driver.py --cdr1-contacts 0,2,4,6 --cdr2-contacts 1,3,5 --cdr3-contacts 2,13,29
```

```
hiv_arms_race_driver — full build (30-aa WT, 0-based, gap=auto)
```

```
[1] landscape: 570 single-aa mutants in 4 ms
[2] gap histogram: {4: 15, 3: 136, 2: 419}
[3] codon-viable: 180/570 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 151
[5] TOP ESCAPE (0-based): pos 2 WT[2]= to A gap=4 frob=True conf=0.60
    WT : TGPCTNVSTVQCTHGIRPVVSTQLLLNGSL
    MUT: TGA CTNVSTVQCTHGIRPVVSTQLLLNGSL
        ^ pos 2 (0-based mutation site)
[6] COUNTER CDR3: WGN S I T K P A V G S engages `>·T·⊠·-·` (full VH Frobenius-OK conf 0.962, thresh
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
[8] COVERAGE: 115/151 gap>=3 mutants retain a CDR1-or-CDR2 contact — 76%
FROBENIUS: all checks closed (B4=T). Driver: imsgct/Vox/hrace_driver.py (30-aa WT, 0-based)
```

The sparse overrides force auto mode (the canonical pins no longer apply), so the gap-4 tier is recomputed from glyph movement  $\times$  local contact count: position 2 sits in CDR1 *and* CDR3 (two contacts) and P $\rightarrow$ A clears  $d \geq 3$  and  $b \geq 3$ , making it the lowest-position gap-4 winner at confidence 0.60. Step [8] reports a *measured* 76%: CDR1  $\cup$  CDR2 here covers only positions 0–6, so the 36 gap $\geq$ 3 mutants outside that window are uncovered.

### 08 — --wt-file FASTA input — exit 0, full 8 steps

```
printf '>custom ep\nAGLTVWPSQNRDEHKCITFVYA\n' > /tmp/hrace_wt.fa && python3
hrace_driver.py --wt-file /tmp/hrace_wt.fa
```

Identical to run 02 in every count (22-aa WT parsed, >header stripped, 418 mutants, histogram {4:86, 3:296, 2:36}, top position 0 A $\rightarrow$ R gap 4 confidence 0.60, coverage 382/382). This demonstrates the file-input lane end-to-end and shows that the per-model default confidence applies the same way to file-loaded WTs.

### 09 — Short 10-aa WT — exit 0; also exercises the $n \leq W$ render path

```
python3 hrace_driver.py --wt AGLTVWPSQN
```

```

hiv_arms_race_driver — full build (10-aa WT, 0-based, gap=auto)
[1] landscape: 190 single-aa mutants in 1 ms
[2] gap histogram: {4: 47, 3: 126, 2: 17}
[3] codon-viable: 59/190 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 173
[5] TOP ESCAPE (0-based): pos 0  WT[0]= to R  gap=4 frob=True conf=0.60
    WT : AGLTVWPSQN
    MUT: RGLTVWPSQN
        ^ pos 0 (0-based mutation site)
[6] COUNTER CDR3: WGENSITKPAVGS engages `>·T·H·H·` (full VH Frobenius-OK conf 0.962, thresh
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
[8] COVERAGE: 173/173 gap>=3 mutants retain a CDR1-or-CDR2 contact — 100%
FROBENIUS: all checks closed (B4=T). Driver: msgct/Vox/hrace_driver.py (10-aa WT, 0-based

```

The 10-aa WT renders through the non-windowed path (caret at column 0, no 60-aa pad) and completes at the auto default confidence 0.60.

#### 04 — Canonical WT, explicit **--gap-model auto**, no overrides — exit 0

```

python3 hrace_driver.py --gap-model auto

hiv_arms_race_driver — full build (30-aa WT, 0-based, gap=auto)
[1] landscape: 570 single-aa mutants in 4 ms
[2] gap histogram: {4: 135, 3: 373, 2: 62}
[3] codon-viable: 180/570 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 508
[5] TOP ESCAPE (0-based): pos 2  WT[2]= to A  gap=4 frob=True conf=0.60
    WT : TGPCTNVSTVQCTHGIRPVVSTQLLLNGSL
    MUT: TGACTIONVSTVQCTHGIRPVVSTQLLLNGSL
        ^ pos 2 (0-based mutation site)
[6] COUNTER CDR3: WGENSITKPAVGS engages `>·T·H·H·` (full VH Frobenius-OK conf 0.962, thresh
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
[8] COVERAGE: 508/508 gap>=3 mutants retain a CDR1-or-CDR2 contact — 100%
FROBENIUS: all checks closed (B4=T). Driver: msgct/Vox/hrace_driver.py (30-aa WT, 0-based

```

Auto mode recovers most of the pinned shape on its own (135 gap-4 mutants versus 6 pinned — auto over-assigns gap 4 to high-movement mutants with  $\geq 2$  contacts, while the pinned table withholds). The winner is the lowest-position gap-4, confidence-0.60 mutant: position 2 P→A (position 2 sits in CDR1 and CDR3 under canonical contacts).

#### 19 — Canonical auto + **--top-conf 0.60** — exit 0

```
python3 hrace_driver.py --gap-model auto --top-conf 0.60
```

Same full 8-step output as run 04 (an explicit 0.60 equals the auto default): top position 2 P→A gap 4 confidence 0.60, coverage 508/508. The tie-break (gap, conf, -pos) picks position 2 because it is the lowest position among the gap-4, confidence-0.60 candidates.

**18 — Fully non-canonical end-to-end attempt — exit 2 (no gap-4 exists under those contacts)**

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA --cdr3 NPGAVG --counter-cdr3
GPGGSG --top-conf 0.50 --counter-conf 0.50 --cdr1-contacts 0,3,6,9 --cdr2-contacts
1,4,7,10 --cdr3-contacts 2,5,8,11,14,17,20
```

```
error: no escape candidate meets --top-conf 0.50
hiv_arms_race_driver — full build (22-aa WT, 0-based, gap=auto)
[1] landscape: 418 single-aa mutants in 3 ms
[2] gap histogram: {3: 242, 2: 176}
[3] codon-viable: 134/418 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 242
```

These contact sets give no position with at least two contacts and a large enough glyph move, so the histogram has no gap-4 tier. Every confidence is  $0.45 + 0.001 \cdot \text{pos}$  (at most about  $0.471$ )  $< 0.50$ . The gate refuses honestly: the landscape was built, the histogram is internally consistent ( $242 + 176 = 418$ ), and there is no eligible candidate. (The error line appears before the header in the capture because the gate's stderr is unbuffered while steps [1]–[4] sit in the stdout buffer.)

**3. Direct function suite (run 10) — import-level demonstration**

Every non-CLI function was exercised directly with concrete values:

```
tuple_of('I'): `t`=J`+`=2`>`=r`<`=c`□`=c`T`=c`€`=J`∃`=p`⊙`=J`⊥`=c`⊞`=Q`⊠`=O
glyph_distance(I,D) = 3.4641      bottleneck_count(I,D) = 3
glyph_distance(A,K) = 4.2426      bottleneck_count(A,K) = 5
is_charged_glyph: D=True K=True R=True I=False G=False
gap_of_pinned(15,I,D) = 4      (the kernel-verified I→D escape pin)
gap_of_pinned(15,I,E) = 3      (CORE4, not a pin → 3)
gap_of_pinned(0,T,D) = 3      (TIER3, charged mutant → 3)
gap_of_pinned(14,G,E) = 2      (BASE → always 2)
gap_of_auto(2 contacts, I→D) = 4      (d=3.4641 ≥ 3.0, b=3 ≥ 3)
gap_of_auto(1 contact, I→E) = 3      (charged-glyph gate)
gap_of_auto(0 contacts, I→D) = 2      (no contact → floor)
counter_engagement_conf(canonical) = 0.962      (hard-coded pin)
counter_engagement_conf(canonical CDR3 vs AAAAAAAAAAAAAA) = 0.5119 (computed)
parse_index_set('0,2,4,6-8') = {0,2,4,6,7,8}; parse_index_set(None) = None
autodetect_contacts(10) = CDR1 {0,2,4,6,8}, CDR2 {1,3,5,7,9}, CDR3 {0,3,6,9}
contact_count(2, canonical) = 2      (pos 2 in CDR1 and CDR3)
choose_wt_codons('GIRP') = ['GGT', 'ATT', 'CGT', 'CCT'] (max synonymous neighbourhood)
_neighbors('ATT') = {F, I, L, M, N, S, T, V}      (8 single-nt neighbours)
vector: len(CDR3+linker+counter)=29 → translate(dna)==payload: True
canonical WT[15] = I      (the I→D escape site)
```

Notes:

- `choose_wt_codons` picks, for each residue, the codon with the largest synonymous single-nucleotide neighbourhood — for example ATT for isoleucine, whose neighbourhood includes the methionine codon (ATG) and the lysine/asparagine class. This is what

the **codon-viable** count measures: a mutant counts as viable if its residue is one single nucleotide away from the chosen WT codon.

- `tuple_of` is defined for all 20 standard amino acids and raises `ValueError` on anything else. This exercises the AA-string domain guard, which main also enforces before the pipeline runs.
- `parse_index_set` raises `ValueError` on malformed input ('0-', 'abc', '0, -1'); main catches it and reports a clean typed error (lanes 20–22).

#### 4. Error paths — all eight validation lanes, exit 2

| #  | Command<br>(abridged)                                                         | Output                                                                            | Guard                                                                                                                                     |
|----|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 11 | <code>--wt ...X...</code><br>(non-standard residue)                           | error: non-standard residue(s) in WT: ['X']                                       | WT domain check, pre-pipeline                                                                                                             |
| 12 | <code>--counter-cdr3</code><br><code>NGISHTKPAVGS (= canonical --cdr3)</code> | error: --counter-cdr3 must differ from --cdr3                                     | payload sanity, pre-pipeline                                                                                                              |
| 13 | non-canonical WT +<br><code>--gap-model</code><br>pinned                      | error: --gap-model pinned requires the canonical WT/CDRs and no contact overrides | mode/entry consistency                                                                                                                    |
| 14 | <code>--cdr1-contacts</code><br>99 (22-aa WT)                                 | error: CDR1 contact index 99 out of range 0..21                                   | contact range, per CDR                                                                                                                    |
| 15 | canonical +<br><code>--top-conf 0.95</code>                                   | error: no escape candidate meets --top-conf 0.95                                  | early canonical confidence check — fires before any pipeline output, since nothing above 0.80 exists in the pinned run (pinned mode only) |
| —  | <code>--wt ''</code> (empty)                                                  | error: empty WT sequence                                                          | empty-input guard                                                                                                                         |

| #  | Command<br>(abridged)            | Output                                                                               | Guard        |
|----|----------------------------------|--------------------------------------------------------------------------------------|--------------|
| 20 | --cdr1-contacts<br>0-            | error: malformed<br>index list for<br>CDR1: '0-'<br>(expected e.g.<br>0,2,4-6)       | list grammar |
| 21 | --cdr1-contacts<br>abc           | error: malformed<br>index list for<br>CDR1: 'abc'<br>(expected e.g.<br>0,2,4-6)      | list grammar |
| 22 | --cdr1-contacts<br>0,-1          | error: malformed<br>index list for<br>CDR1: '0,-1'<br>(expected e.g.<br>0,2,4-6)     | list grammar |
| 23 | --wt-file<br>/tmp/does_not_exist | error: cannot<br>read wt file<br>...: [Errno 2] No<br>such file or<br>directory: ... | I/O guard    |

All refusals are clean: one line to stderr, exit 2, no partial pipeline output — except run 18, where the gate sits after steps [1]–[4] by design. Lanes 20–23 cover malformed contact lists and a missing --wt-file: `parse_index_set` raises `ValueError` on a bad index list, `read_wt_from_file` raises `OSError` on an unreadable path, and both are caught in `main` with the same typed one-line exit-2 semantics as the other lanes.

## 5. What the results show

**The driver is a checksummed, self-certifying pipeline, and the canonical run is its fixed point.** Run 01 closes all eight steps with every internal assertion holding. The gap histogram {4:6, 3:187, 2:377} is not merely reported — it is *asserted* equal to `CANONICAL_HIST`. The six gap-4 mutants are asserted to be exactly the kernel-verified CORE4 pins. The top escape is asserted to be position 15, I→D, gap 4, with the exact mutant sequence. Coverage is asserted 193/193. The final line `FROBENIUS: all checks closed (B4=T)` is therefore not a slogan; on this run it is backed by six executed assertions. The glyph algebra is doing the scoring: I→D reads a distance of 3.4641 with 3 bottleneck disagreements, which is precisely the shape `gap_of_auto` would also promote to gap 4 when two or more contacts are present. The pinned table and the glyph rule agree on the canonical escape — which is the whole point of the pin.

**The two gap models are the same algebra read two ways.** Pinned reads the position class from the kernel-verified partition and applies only the charged-glyph test inside TIER3. Auto reads both class and severity directly off the glyph tuple plus the local contact count. Run

04 shows auto recovering most of the pinned shape on its own (135 gap-4 mutants versus 6 pinned — auto over-assigns gap 4 to high-movement mutants with  $\geq 2$  contacts, while the pinned table withholds). Run 10 shows the boundary cases of each rule in isolation: CORE4 $\rightarrow$ 4 only for the six pins, TIER3 $\rightarrow$ 3 if charged, BASE $\rightarrow$ 2 always; auto $\rightarrow$ 4 if  $d \geq 3$  and  $b \geq 3$  at  $\geq 2$  contacts,  $\rightarrow 3$  if charged or  $d \geq 2.5$  at 1 contact,  $\rightarrow 2$  otherwise. Both readings terminate and both close B4=T on the same 570-mutant landscape.

**The thresholds are real gates, not formatting.** --top-conf changes *who is eligible* (run 05: widens eligibility, same winner), and run 19 shows it can even change the *identity* of the winner once gap-4 mutants at confidence 0.60 enter the pool (the top becomes position 2, P $\rightarrow$ A). --counter-conf moves the engagement verdict between “engages” and “does NOT engage” on an unchanged computed confidence (runs 01/06), and a non-canonical counter gets a *computed* 7-bottleneck confidence (run 07: 0.512). Run 18 shows the gate refusing when the landscape genuinely contains nothing eligible (no gap-4 tier, all confidences  $\leq 0.471 < 0.50$ ).

**The vector build is an exact codec, not a lookup.** Step [7] assembles CDR3 + linker + counter, encodes it using the first codon of each residue, and asserts `translate(dna) == payload` — 29 amino acids  $\rightarrow$  87 nt canonically, 33 amino acids  $\rightarrow$  99 nt with the custom counter, both round-tripping. The codon-viable count (180/570 canonical, 134/418 custom) is the same machinery applied to the WT side: a mutant counts as low-risk if its residue sits in the single-nucleotide neighbourhood of the max-neighbourhood WT codon (run 10: `_neighbors('ATT')` gives 8 residues).

**Every input lane works, and every failure is typed.** --wt (01, 02, 17), --wt-file FASTA with header stripping (08), contact overrides with ranges (03), explicit gap model (04, 19), short sequences through the non-windowed render path (09), and all eight validation refusals with clean exit-2 semantics (11–15, empty WT, 20–23). --help documents the full surface (16).

## 6. Findings — where the suite exposes tension

1. **The default --top-conf is model-dependent.** Auto-mode confidences top out at 0.60 (gap-4) and  $0.45 + 0.001 \cdot \text{pos}$  otherwise. A single hard default would be wrong for one of the two models; the per-model default (auto  $\rightarrow$  0.60) makes the tool’s own auto mode reachable out of the box.
2. **The canonical convention guard is scoped to the model it encodes.** The guard fires only under `gap_mode == 'pinned'`. Under auto gaps the top is (2, 'A', 4) (runs 04, 19), which is a legitimate auto-mode answer; scoping the guard to pinned mode means canonical + auto completes and reports the auto answer.
3. **Coverage 100% under auto is by construction — and run 03 proves the distinction matters.** `autodetect_contacts` puts every even position in CDR1 and every odd in CDR2, so CDR1 $\cup$ CDR2 is the whole epitope and step [8] cannot report below 100% for autodetected inputs (runs 02, 04, 09, 17: 382/382, 508/508, 173/173, 382/382). It is only informative on canonical (193/193, where the contacts are real) or explicit-override inputs — and run 03 is the witness: sparse overrides covering only positions 0–6 yield a *measured* 115/151 — 76%, with the 36 uncovered gap $\geq 3$  mutants living outside the contact window.
4. **stderr/stdout interleave is an artifact, not a bug.** In run 18 the error line appears *before* the pipeline header in captured output; the gate executes after steps [1]–[4] — unbuffered stderr simply overtakes buffered stdout under `2>&1`.

---

## 7. Reproduce

```
cd /home/mrnob0dy666/imsct/Vox
python3 hrace_driver.py                                # 01 canonical pinned (fixed point)
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKITFVYA    # 02 auto, default conf 0.60, full
python3 hrace_driver.py --gap-model auto              # 04 canonical auto
python3 hrace_driver.py --counter-conf 0.99           # 06 engagement demoted
python3 hrace_driver.py --counter-cdr3 AAAAAAAAAAAAAA # 07 computed conf
printf '>custom ep\nAGLTVWPSQNRDEHKITFVYA\n' > /tmp/hrace_wt.fa
python3 hrace_driver.py --wt-file /tmp/hrace_wt.fa     # 08 FASTA lane
python3 hrace_driver.py --wt-file /no/such/file        # 23 typed I/O error, exit 2
python3 hrace_driver.py --cdr1-contacts 0-            # 20 typed list error, exit 2
```

Full transcripts (all 22 runs with exit codes): /tmp/hrace\_results3.txt.